MINHAJ UNIVERSITY LAHORE

School of Software Engineering

TrustLens

AI-Powered Seller Trust Verification System for Online Marketplaces

A Final Year Project Report submitted in partial fulfilment of the requirements for the degree of **Bachelor of Science in Software Engineering**

Haseeb Ur Rehman — 2022F-mulbsswe-043 **Muhammad Muttayab** — 2022F-mulbsswe-012

Supervisor: **Mam Misbha** School of Software Engineering, Minhaj University Lahore

2026

Certificate of Approval

This is to certify that the Final Year Project titled **"TrustLens — AI-Powered Seller Trust Verification System for Online Marketplaces"** has been carried out and completed by **Haseeb Ur Rehman (2022F-mulbsswe-043)** and **Muhammad Muttayab (2022F-mulbsswe-012)** under my supervision, and that it meets the requirements of the School of Software Engineering, Minhaj University Lahore, for the degree of Bachelor of Science in Software Engineering.

Supervisor: Mam Misbha	Signature: _____
FYP Coordinator: Muhammad Kamran	Signature: _____
Head of Department: Imran Akther	Signature: _____
External Examiner: _____	Signature: _____

Check this page's exact wording against your institution's current required certificate format before binding — this was written from the standard FYP proposal submission form, not a signed institutional template.

Declaration of Originality

We, Haseeb Ur Rehman and Muhammad Muttayab, declare that this Final Year Project report is our own work, carried out under the supervision of Mam Misbha at the School of Software Engineering, Minhaj University Lahore. Where the design draws on pretrained third-party models, open-source libraries, or published research, each is explicitly cited in Chapter 2 and the References section. No part of this report has been submitted for any other degree or qualification.

Haseeb Ur Rehman	Signature: _____
Muhammad Muttayab	Signature: _____

Acknowledgement

We are grateful to our supervisor, Mam Misbha, for her guidance throughout this project, and to Muhammad Kamran, FYP Coordinator, for his support during the proposal and evaluation stages. We also acknowledge the contribution of Zohaib Khan, who was contracted to build the software system documented in this report under our direction, and whose engineering work this thesis describes and evaluates.

Abstract

Online marketplace buyers routinely face fraud from fake seller profiles, stolen product photos, throwaway phone numbers and email addresses, multiple accounts created from a single device, and listings that are "too good to be true," with no reliable way to judge a seller before paying. **TrustLens** addresses this by combining seven independent verification signals — identity (CNIC optical character recognition plus selfie-to-document face matching), phone, email, device, network, account behaviour, and AI-assisted listing analysis — into a single, explainable **Trust Score** between 0 and 100, with a risk band and the specific reasons behind it. The central design commitment is that the score is computed by a deterministic, versioned rule engine, never directly by an AI model: AI components (OCR, face matching, a large-language-model listing analyser, and a custom-trained scam-listing classifier) supply typed evidence with a confidence value, which the rule engine converts into points. This makes every score reproducible and appealable by construction. The system was built as a modular monolith — a thin, synchronous FastAPI backend, an asynchronous ARQ worker for AI-heavy processing, a Next.js frontend, and a framework-free domain package enforcing clean-architecture boundaries — and runs entirely offline at zero recurring cost, a deliberate constraint for a two-student, self-funded FYP. As of this report, the core seller journey (registration through identity/phone/email verification, score computation, and appeal resolution) works end to end and is covered by 330 passing automated tests; the administrative panel and public deployment remain partially complete, and no accuracy figures exist yet for OCR or face matching against real identity documents, since collecting a consented real-image test set was assigned as ongoing student work rather than an engineering task. A cross-corpus evaluation of the custom scam-listing classifier revealed a significant, honestly-reported weakness — strong performance (0.906 F1) on its own training-domain data but poor performance (catching only 1 of 7 real Pakistani marketplace scam listings in a manual check) on the product's actual target text — which is why the classifier is not yet enabled as a live scoring input. This report documents the system's requirements, design, implementation, and evaluation in full, including these limitations, on the view that an honestly reported partial result defends better at viva than an inflated claim.

Keywords: trust score, seller verification, explainable AI, fraud detection, identity verification, marketplace security, clean architecture.

Table of Contents

The paginated contents table is generated in Project_Thesis.docx. In this HTML edition, use the chapter headings below to navigate.

List of Figures

#	Figure	Page
1	System context diagram	—
2	Layered architecture diagram	—
3	Entity-relationship diagram (core tables)	—
4	Use case diagram	—
5	Trust Score computation activity diagram	—
6	Identity verification sequence diagram	—
7	Partner score-read sequence diagram, with degraded path	—
8	Refresh-token reuse-detection sequence diagram	—
9	Verification status lifecycle (state diagram)	—
10	Deployment diagram (as designed)	—
11	Landing page (live screenshot)	—
12	Trust Score breakdown pages, three risk bands (live screenshots)	—
13	Public buyer-facing seller page (live screenshot)	—

Page numbers are populated automatically when the Table of Contents field is updated in Word (press F9 → "Update entire table") — see the handover note.

List of Tables

#	Table
1	Fraud pattern → defence mapping
2	Related-systems comparison
3	Trust Score signal reference (v1 model)
4	Hard-cap table
5	Objectives-achieved scorecard
6	Test coverage summary (specified vs. executed)
7	Defect log summary
8	Future-work priority table

List of Abbreviations

Abbreviation	Meaning
CNIC	Computerized National Identity Card
OCR	Optical Character Recognition
API	Application Programming Interface
RBAC	Role-Based Access Control
JWT	JSON Web Token
LLM	Large Language Model
IDOR	Insecure Direct Object Reference
ARQ	Async Redis Queue
SRS	Software Requirements Specification
SDD	Software Design Document
ADR	Architecture Decision Record
ONNX	Open Neural Network Exchange (runtime format)
FYP	Final Year Project

Chapter 1 — Introduction

1.1 Background

Online marketplaces (OLX-style classifieds, rental and property portals, second-hand vehicle sites, and freelance platforms) connect strangers for a transaction with very little institutional trust infrastructure between them. Unlike a licensed retailer, an individual seller on such a platform carries almost no verifiable history a buyer can inspect before paying.

1.2 Problem statement

The problem decomposes into five concrete, recurring fraud sub-problems, each named directly in the approved project proposal [12]:

1. **Fake seller profiles** — a seller's claimed identity cannot be verified by a buyer.
2. **Stolen product photos** — images lifted from other listings or the wider web misrepresent what is actually for sale.
3. **Throwaway phone numbers and email addresses** — contact channels that stop responding after payment, with no cost to abandon.
4. **Multiple accounts from one device** — a banned or flagged seller re-enters under a new identity from the same hardware.
5. **Unrealistic, "too good to be true" listings** — pricing and language patterns typical of advance-fee and non-delivery scams.

No existing mechanism combines checks against all five into one buyer-facing signal that explains itself.

1.3 Motivation

The two most common existing responses — a simple phone-OTP gate (as many marketplaces already implement) or an unstructured star-rating/review system — each address only a narrow slice of the problem and neither is explainable. A phone OTP proves control of a number, not identity, and not listing honesty. A star rating requires a completed transaction history to exist before it says anything useful, which is exactly the information a first-time buyer lacks. This motivated a design that (a) checks several

independent signals rather than one, and (b) always shows its reasoning, rather than asking a buyer to trust an opaque score.

1.4 Objectives

Objective	Measurable success criterion
O1 — Verify seller identity	A seller can complete CNIC OCR + selfie face-match verification, and the result contributes to the score
O2 — Verify contact channels	Phone OTP and email link verification work end to end and feed the score
O3 — Detect device/network fraud patterns	Multiple accounts sharing one device, and VPN/proxy connections, are detected and scored
O4 — Analyse listing content with AI	Listing text is analysed by an LLM and a custom classifier for scam indicators
O5 — Produce one explainable score	A single 0–100 score with a risk band and a full, reconstructable per-signal breakdown exists for every seller with sufficient evidence
O6 — Support appeals	A seller can appeal a specific score, and a human reviewer can resolve it with a recorded reason
O7 — Expose a public API	A partner marketplace can fetch a seller's score via a documented, authenticated, rate-limited REST API
O8 — Operate at zero recurring cost	The full stack runs offline, and the designed deployment target uses only free-tier managed services

Objectives are scored honestly against these criteria in Chapter 7 (§7.3) — not every one was fully achieved by the time of this report.

1.5 Scope

In scope: the eight features listed in the approved proposal (identity, phone, email, and device checks; AI listing analysis; the scoring engine; an admin/appeal panel; a public API) — see the SRS (`03-SRS.md`) §1.2 for the full in/out-of-scope tables.

Explicitly out of scope, per the proposal's own stated future scope: reverse-image stolen-photo detection, chat-based scam detection, direct marketplace integrations, seller reputation history over time, and a mobile application. Also out of scope, and stated here rather than discovered later: any claim of government-grade identity verification (no public NADRA API exists at FYP access level), and validated real-world OCR/face-match accuracy figures (no consented real test-image set exists yet).

1.6 Limitations, stated up front

Stating limitations before the results chapter, rather than only at the end, is a deliberate choice so a reader calibrates expectations correctly from the start rather than being surprised later:

- No real CNIC/selfie test images have been benchmarked yet — accuracy claims for OCR and face matching are architectural/contractual (the engines are correctly wired and pass synthetic-fixture tests), not empirically validated against real documents.
- The custom-trained scam-listing classifier performs poorly on the product's actual target text (see §7 for the measured numbers) and is deliberately not yet a live scoring input.
- The administrative panel is functionally incomplete — only the identity/appeal review queue and scoring-model view/simulate/activate flows work end-to-end.
- Multi-factor authentication for staff accounts is designed but not implemented.
- No public deployment exists at the time of writing; the system runs correctly locally via Docker.

1.7 Thesis organisation

Chapter 2 reviews related systems and research. Chapter 3 condenses the requirements analysis. Chapter 4 condenses the system design. Chapter 5 describes what was actually implemented and the real challenges encountered. Chapter 6 reports testing and evaluation with full honesty about what was executed versus only specified. Chapter 7 presents results, screenshots, and an honest objectives-achieved scorecard. Chapter 8 concludes and sets out future work.

Chapter 2 — Literature Review

2.1 Trust and reputation systems in e-commerce

The general problem of establishing trust between strangers transacting online is well studied. Resnick and Zeckhauser's foundational work on eBay's feedback mechanism, and subsequent surveys and mechanism-design analyses of trust and reputation systems in e-commerce [1][2][3][11], establish that reputation systems broadly fall into two families: **outcome-based** systems (star ratings, feedback scores, accumulated after a transaction completes) and **attribute-based** systems (verified credentials, checked before a transaction). A comprehensive survey of e-commerce trust and reputation mechanisms [2] compares systems on utility, usability, performance, and effectiveness, and notes that outcome-based systems structurally cannot help a *first-time* buyer evaluating a seller with no transaction history — exactly the gap TrustLens targets by being attribute-based (identity, contact, device, and listing checks that exist before any transaction occurs) rather than outcome-based.

2.2 Automated fraudulent-text detection

The AI listing-analysis component (F5) draws directly on research into detecting fraudulent job postings, a closely related text-classification problem: distinguishing genuine adverts from scam/fraudulent ones based on language patterns (urgency, unrealistic offers, requests to move off-platform). Published work on the **Employment Scam Aegean Dataset (EMSCAD)** [4][5] establishes that classical and ensemble machine-learning classifiers (Random Forest, and deep architectures combining CNN/LSTM layers over text embeddings) can achieve high accuracy — commonly cited in the 90%+ range — **on their own training-domain data**. TrustLens's own scam-listing classifier is trained partly on EMSCAD and a second proxy fraud corpus (DIFrauD), which is precisely why its strong own-corpus F1 score (0.906, Chapter 6) closely matches the accuracy ranges reported in this literature — and precisely why the same literature's implicit assumption (that a classifier's own-corpus performance predicts its real-world performance) was worth testing rather than trusting, because the cross-corpus results in Chapter 6 show that assumption failing badly for this specific domain transfer (English job adverts → Pakistani marketplace listings).

2.3 Identity document verification and forgery detection

Automated identity-document verification (the OCR half of Feature F1) is an active research and commercial area. Recent work on document-forgery detection combines OCR-based preprocessing with

convolutional and transformer architectures to detect manipulated identity documents [6][7][8], and a 2026 systematic survey of identity-document attack and detection methods [9] documents that production KYC (Know Your Customer) systems increasingly face AI-generated and synthetically forged documents as an emerging threat class. TrustLens's design deliberately does not attempt forgery *detection* in this sense — it performs OCR field extraction and plausibility checks (expiry date, format, quality gates) rather than adversarial forgery detection, which is a materially harder research problem placed outside this FYP's scope, and this report says so rather than implying a stronger guarantee.

2.4 Face-recognition accuracy and demographic fairness

The face-match component's design (the deliberate 0.55–0.70 "manual review" band, rather than a single hard threshold) is directly informed by the U.S. National Institute of Standards and Technology's Face Recognition Vendor Test, Part 3: Demographic Effects [10], which found that the majority of tested face-recognition algorithms exhibit measurable false-positive and false-negative rate disparities across demographic groups, with the gap between the best- and worst-performing algorithms spanning false-negative rates from under 1% to as high as 99% depending on the demographic slice and algorithm tested. This is the specific research basis for TrustLens's design decision (SDD §8.3) to route borderline face-match results to a human reviewer rather than an automatic rejection — a system-level mitigation for a known, externally-validated technical limitation, rather than an assumption that the underlying face-matching library is unbiased.

2.5 Related systems comparison

System / approach	Mechanism	Explainable to the buyer?	Works for a first-time seller?	Combines identity + behaviour + content signals?
eBay-style feedback score	Outcome-based, post-transaction ratings	Partially (aggregate only)	No — needs transaction history	No
Marketplace phone-OTP gate (e.g. many OLX-style sites)	One contact-channel check	No	Yes	No — single signal only
Commercial KYC/identity-verification vendors (document + selfie check, e.g. the class of product Jumio/Onfido represent)	Attribute-based, identity only	Rarely exposed to the end buyer	Yes	No — identity only, not combined with behavioural/content signals
Star-rating + written reviews	Outcome-based, subjective	Partially	No	No

System / approach	Mechanism	Explainable to the buyer?	Works for a first-time seller?	Combines identity + behaviour + content signals?
TrustLens	Attribute-based, 7 independent signal families, deterministic scoring	**Yes — full per-signal breakdown**	**Yes — before any transaction**	**Yes**

2.6 Identified gap

Gap in existing approaches	How TrustLens addresses it
No single system combines identity, contact, device, network, behavioural, and content signals into one score	Seven independent signal families feed one deterministic scoring policy (Chapter 4)
Reputation scores are typically either a black-box AI opinion or an unexplained aggregate	Every score persists a full, reconstructable per-signal breakdown at computation time — explainable by construction, not by asking an AI to explain itself
Outcome-based systems cannot help a first-time buyer	TrustLens scores a seller before any transaction occurs, using verifiable attributes
Face-matching fairness disparities are rarely mitigated at the system-design level in student projects	A deliberate borderline band routes to human review rather than an automatic decision

Chapter 3 — Requirement Analysis

Condensed from the full Software Requirements Specification, `03-SRS.md`. Requirement IDs (e.g. FR-SCORE-001) refer to that document, which remains the authoritative, complete version.

3.1 Elicitation method

Requirements were derived from the approved FYP proposal form (a four-page document specifying objective, scope, target users, technology stack, and features) plus 15 explicit assumptions filling gaps the proposal left open (auth mechanism, risk-band thresholds, storage location for sensitive documents, and others) — each recorded with its blast radius if a supervisor overrules it, so no assumption is silently load-bearing (SRS §2.6).

3.2 User classes

Buyer (anonymous or registered), Seller, Reviewer, Admin, and Partner marketplace (a machine client). Full permission matrix: SRS §2.3 and `docs/09-user-roles.md`.

3.3 Functional requirement summary

Module	Requirement count	Representative requirement
Authentication	10	Refresh-token reuse detection revokes the whole token family
Identity verification	10	A borderline face-match result routes to manual review, never an automatic fail
Phone / Email	7	OTP send-rate limited to 3/hour, 10/day
Device / Network	4	More than 2 accounts sharing a device is a negative signal
Listing / AI analysis	5	Listing text is treated as untrusted data, never as LLM instructions
Trust Score engine	11	Unavailable signals are excluded from the denominator, never scored as zero
Appeals	5	A reviewer cannot override a score directly, only an admin can
Admin panel	6	One requirement (seller search) is explicitly flagged as not yet implemented
Public API	7	No PII field may exist in any public response schema

70 functional requirements in total, each mapped to at least one test case in the traceability matrix (SRS Appendix A).

3.4 Use case model

The full use-case diagram and three detailed specifications (identity verification, public score lookup, appeal resolution) are in SRS §5. The single most architecturally significant use case is **Complete identity verification**, because its alternative and exception flows (a borderline face-match score, an OCR engine crash on malformed input) are where the system's fairness and robustness design actually shows up.

3.5 Non-functional requirements summary

Category	Headline requirement
Performance	Public score read p95 < 150ms (cache hit); full identity verification 30–60s (distinct from the read latency — see the corrected "within seconds" claim, SRS §6.1)
Safety	AI output never directly sets the score (NFR-SAFE-02)
Security	MFA mandatory for staff roles (**designed, not yet implemented** — see §5.4)
Cost	The entire system must be demonstrable at $0/month (NFR-COST-01)

3.6 Feasibility study

Dimension	Assessment
Technical	All mandated technologies (Next.js, FastAPI, PostgreSQL, Redis, OpenAI/Claude, OpenCV/EasyOCR, DeepFace) are mature, well-documented, and were successfully integrated
Economic	Zero licensing cost; free-tier hosting is designed and partly measured (worker image RAM footprint benchmarked at 323MB)
Operational	A two-person team with a 44-day (software) + 9-day (documentation) schedule; ~51% complete at the time of this report, with the highest-risk remaining work (real accuracy data collection) explicitly assigned to the students
Legal / ethical	Biometric data processing requires explicit versioned consent (implemented); no government identity-database access is claimed or attempted
Schedule	On the sequencing described in `REMAINING.md`, deployment and full admin-panel completion are the largest remaining schedule risks

Chapter 4 — System Design

Condensed from the full Software Design Document, `04-SDD.md`, which contains the complete diagram set (20+ Mermaid diagrams), full data dictionary, and design-decision rationale table. That document remains authoritative for implementation detail.

4.1 Architecture overview

TrustLens is a **modular monolith with an out-of-process worker**, not a microservices system — a deliberate choice for a two-person team and one bounded context (SDD §2.4). Four deployable units share two codebases:

- `apps/web` (Next.js) — all human-facing UI: public score lookup, seller portal, admin panel.
- `apps/api` (FastAPI) — thin, synchronous HTTP surface; never runs AI-heavy work in the request path.
- `apps/worker` (ARQ over Redis) — OCR, face matching, LLM calls, classifier inference, and score computation, all asynchronous.
- `apps/ml` — an offline training pipeline for the scam-listing classifier, versioned separately from the serving path.

Both `apps/api` and `apps/worker` depend on a framework-free domain package (`packages/py/trustlens-core`) that defines entities, use cases, and ports (Protocol interfaces) with zero imports of FastAPI, SQLAlchemy, or any AI vendor SDK — enforced in CI, not just by convention.

4.2 Data design

The database holds 40 PostgreSQL tables. The two that matter most for the product's central claim are `trust_scores` (the score itself, immutable and append-only, so history is never lost) and `trust_score_signals` (one row per contributing signal per score, carrying its status, points, and a human-readable reason) — together, these are what make a score explainable without recomputation. A partial unique index guarantees a seller can never have more than one *current* score, enforced by the database itself rather than application discipline.

4.3 The Trust Score algorithm

Fourteen signals across seven categories sum to 100 possible points. Three properties distinguish this from a naive weighted average:

1. **Signals that could not be evaluated are excluded from both the numerator and denominator**, never scored as a failure — protecting a seller from being penalised for a third-party outage that was not their fault.
2. **Critical-signal failures apply a hard cap** on the final score regardless of how well every other signal scored — closing the specific exploit where a seller passes every easy check while failing the one hard-to-fake check (identity face match) and would otherwise still land in the "low risk" band.
3. **Insufficient total evidence yields a fourth state, UNKNOWN**, distinct from and never confusable with HIGH risk — a brand-new honest seller is not the same thing as a proven scammer, and the design treats them differently.

4.4 Security design highlights

Password hashing (argon2id), envelope-encrypted CNIC/DOB fields, refresh-token rotation with reuse detection, mandatory object-level ownership checks (preventing IDOR), and a deny-by-default routing assertion that fails the application's own startup if any non-public route lacks an authorization dependency. The most domain-specific control addresses prompt injection: because a seller's own listing text is passed to an LLM, that text is treated as untrusted data whose output is schema-validated and range-checked, never as instructions the model could follow — the worst a successful injection can achieve is a wrong label on one 10-point signal, visible and appealable.

4.5 Key design decisions

The single design decision most worth defending at viva is **ADR-0002: a deterministic rule engine computes the score; AI supplies evidence, never the verdict.** The alternative — letting an LLM or classifier output the score directly — was rejected because it is neither reproducible (the same inputs could yield different scores on a re-ask) nor appealable (a reviewer cannot audit "the model said so"). The full design-decisions table with every alternative considered is SDD §12.

Chapter 5 — Implementation

5.1 Environment and tools

Layer	Technology
Frontend	Next.js 15 (App Router), React 19, TypeScript (strict), Tailwind CSS 4
Backend	Python 3.12, FastAPI, SQLAlchemy 2.0 (async), Alembic, Pydantic v2
Worker	ARQ over Redis
Database	PostgreSQL 16, Redis 7
AI	OpenAI / Anthropic (Claude) API; EasyOCR + DeepFace, and a lighter RapidOCR + InsightFace (ONNX) pair kept side-by-side for benchmarking; a custom-trained scam-listing classifier
Tooling	pnpm + Turborepo (JS), uv (Python), Docker, Postman, Git/GitHub, Figma

5.2 What was actually built, by layer

- **Domain (`trustlens-core`):** entities, use cases, ports, and policies for auth, verification, listings, scoring, and appeals — fully implemented, 95%-coverage-gated in CI.
- **Backend (`apps/api`):** 50 API paths / 58 operations across the internal (`/api/v1`) and public (`/public/v1`) surfaces, all documented via OpenAPI.
- **Worker (`apps/worker`):** OCR, face-match, LLM listing-analysis, scoring, webhook-delivery, and maintenance jobs, all idempotent and retried with bounded `max_tries`.
- **Frontend (`apps/web`):** the full seller journey (registration through appeal), the public buyer score page, and a partial admin panel (identity/appeal review queue and scoring-model management work; seller search/detail and the audit-log viewer do not yet).
- **ML (`apps/ml`):** a versioned training pipeline for the scam-listing classifier, with its own import-linter contract so training code can never reach production.

5.3 The scoring algorithm in practice

Pseudocode for the exact algorithm, including the normalisation and hard-cap steps, is in SDD §8.1. It is implemented as pure, dependency-free code specifically so it can be unit-tested exhaustively — every

signal-status combination, every boundary value at a risk-band edge (39 vs. 40, 69 vs. 70), and every hard-cap scenario is individually tested (Chapter 6).

5.4 Challenges encountered and how they were resolved

A genuine record of real problems hit during development, not retrospectively softened:

Challenge	What went wrong	How it was resolved
Silent job-name mismatch	Every background job (OCR, face match, notifications, OTP, scoring) was enqueued under names that didn't match ARQ's auto-registered function names — jobs were accepted by Redis and silently dropped by the worker. Nothing had ever actually run; demo data only looked healthy because seed data writes scores directly to the database.	Explicit `arq.func(..., name=...)` registration, pinned by a test that scans every call site for an unregistered name
A startup safety check that checked almost nothing	The route-guard assertion meant to fail the application's own boot on any unauthenticated route was walking placeholder routes rather than the real, fully-registered tree, due to a FastAPI version behaviour change — it would only have warned, never failed, exactly where it needed to fail	Rewritten to walk the real route tree after full registration
The public buyer page 404'd for every seller	The one screen the whole product's value proposition depends on returned 404	A new first-party internal route was added, deliberately reusing the partner API's own use case and response model rather than giving the web app its own partner key (which would let a revoked key take down the demo's own opening screen)
Docker images could not build at all	Discovered only by attempting the build for the first time — three independent causes across the API, web, and admin images	Each cause (incomplete build context, a missing Next.js config flag, a hardcoded wrong entrypoint) fixed independently
Secrets were inside the Docker build context	No `.dockerignore` existed; a 236MB build context included `.env.production` with live database and JWT secrets, which a `COPY . .` step would have baked permanently into an image layer	`.dockerignore` added; context reduced to 3MB; verified no secret reaches any built image
The scam classifier looked successful until tested cross-domain	0.906 F1 on its own training-corpus split looked like a finished feature	A deliberate cross-corpus test (not originally planned) revealed the real number — 1 of 7 real target-domain scams caught — which is why the classifier stays out of live scoring
Browsing the demo accounts silently destroyed the demo itself	Found while capturing screenshots for this very report: opening any seller dashboard fires a device-fingerprint check that triggers a real score recomputation; since the seed never stores a selfie file, that recompute fails identity for every demo seller and overwrites the intentionally distinct LOW/MEDIUM/HIGH scores with an identical capped HIGH score — a demo that quietly breaks itself the moment more than one seller account is opened	Data restored for this report's screenshots; the underlying trigger (§6 DEF-14) is still open and must be rehearsed around, or fixed, before a live viva demonstration

Six more logged defects, all fixed, plus one still-open defect found live while producing this report, are in the full defect log (`05-Test-Cases.md` §6).

Chapter 6 — Testing and Evaluation

Condensed from `05-Test-Cases.md`, which remains authoritative and carries the same honesty banner reproduced here: this chapter states two different numbers side by side and never lets one be mistaken for the other.

6.1 What was actually executed vs. only specified

51 test cases were specified across 8 modules, derived directly from the functional requirements. **26 (about 51%) have real, dated evidence behind them** — either the project's own continuously-run automated CI suite (330 passing Python unit/integration tests, 7/7 Playwright E2E specs, 16/16 automated security checks, all last measured 22 August 2026) or a manual verification pass performed while producing this documentation package. The remaining 25 are a legitimate test design that has not yet been executed. **No case in this report is presented as passed without dated evidence.**

Module	Specified	Executed	Pending
Auth	8	4	4
Identity verification	7	0	7
Phone / Email	4	0	4
Trust Score engine	10	7	3
Appeals / Admin	4	3	1
Public API	6	3	3
Security	6	4	2
AI/ML evaluation	6	5	1
Total	**51**	**26**	**25**

6.2 The Trust Score engine — the highest-confidence part of the system

Because the scoring policy is pure, dependency-free code, it carries the most thoroughly executed test coverage in the project: all-PASS → 100, all-FAIL → 0, an unavailable signal excluded from the denominator (confirmed: a seller with one 10-point signal unavailable still scores 100, not 90, when everything else

passes), a hard cap correctly clamping a would-be-85 score to 25 when identity face match fails, and exact risk-band boundary behaviour at 39/40 and 69/70.

6.3 The scam classifier — a result worth reporting exactly as measured

Test	Precision	Recall	F1
Held-out split of its own training corpora	0.918	0.894	**0.906**
A fraud corpus never trained on	0.698	0.665	**0.681**
A second, different unseen corpus	0.320	0.714	**0.442**
Real Pakistani marketplace ads (manual check)	—	**1 of 7 caught**	—

Given the listing "*Send advance payment via easypaisa. WhatsApp me, no calls. Today only, hurry!*" — language that reads as an obvious scam pattern to a human — the classifier returned **genuine, p=0.13**. This is not a minor calibration issue; it is a domain-transfer failure, and it is the reason the classifier is deliberately excluded from live scoring pending real target-domain training data.

6.4 Security testing

16 of 16 automated security checks pass (`make prod-security`), including a fixed defect where account lockout was scoped per-account only, letting one IP spray one password across unlimited email addresses. A manual RBAC review and an independent penetration test remain out of scope for this FYP's test plan and are explicitly listed as such rather than silently omitted.

6.5 What remains untested

No load test has been run against the deployed latency targets (nothing is deployed). No real CNIC or selfie image has ever been processed by either OCR/face-match engine pair. These are stated as open items in Chapter 1 and revisited honestly in Chapter 8, not treated as resolved.

Chapter 7 — Results and Discussion

7.1 The finished product

The core seller-to-buyer journey works end to end in a running local deployment: a seller registers, verifies their email, signs in, completes identity/phone/email/device verification, receives an explainable Trust Score, and can appeal it; a reviewer can decide the appeal and trigger a recomputation; a buyer can look up that score with no account at all.

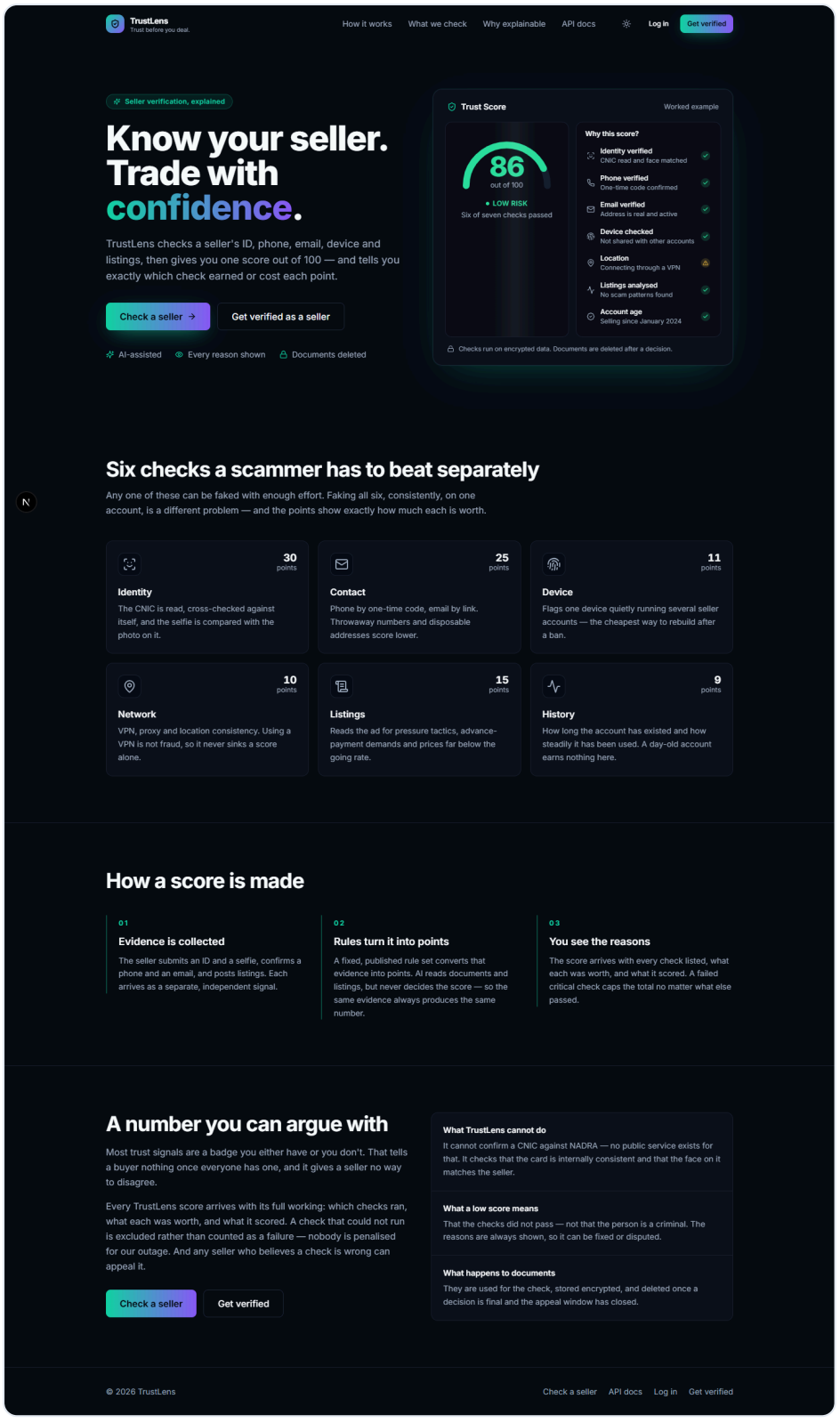


Figure 11 — The landing page states the product's core claim directly: a Trust Score with a full, worked-example breakdown of why it is what it is, visible before a visitor even creates an account.

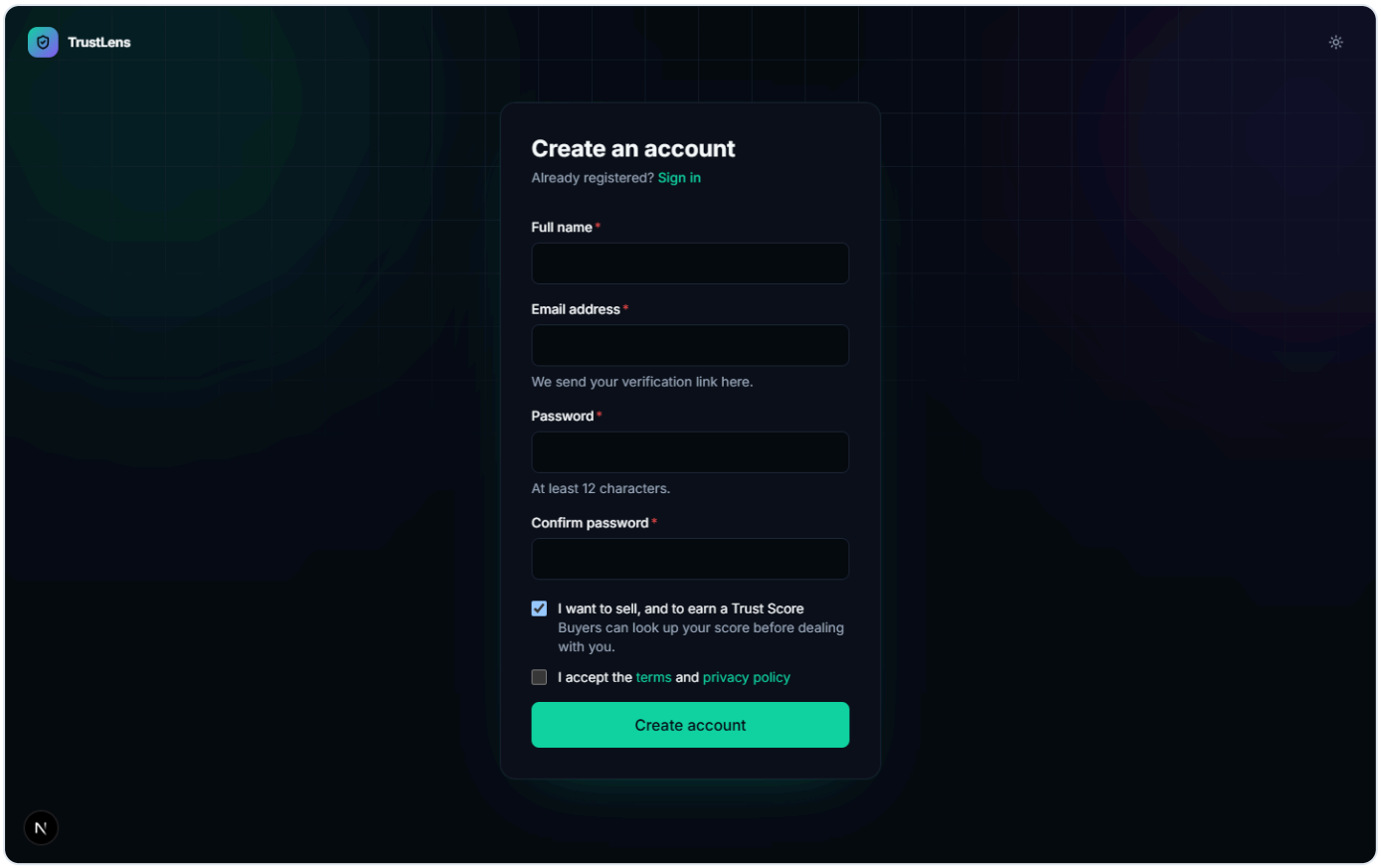


Figure — Registration collects the minimum needed to create an account and immediately states the password policy and the seller/buyer choice.

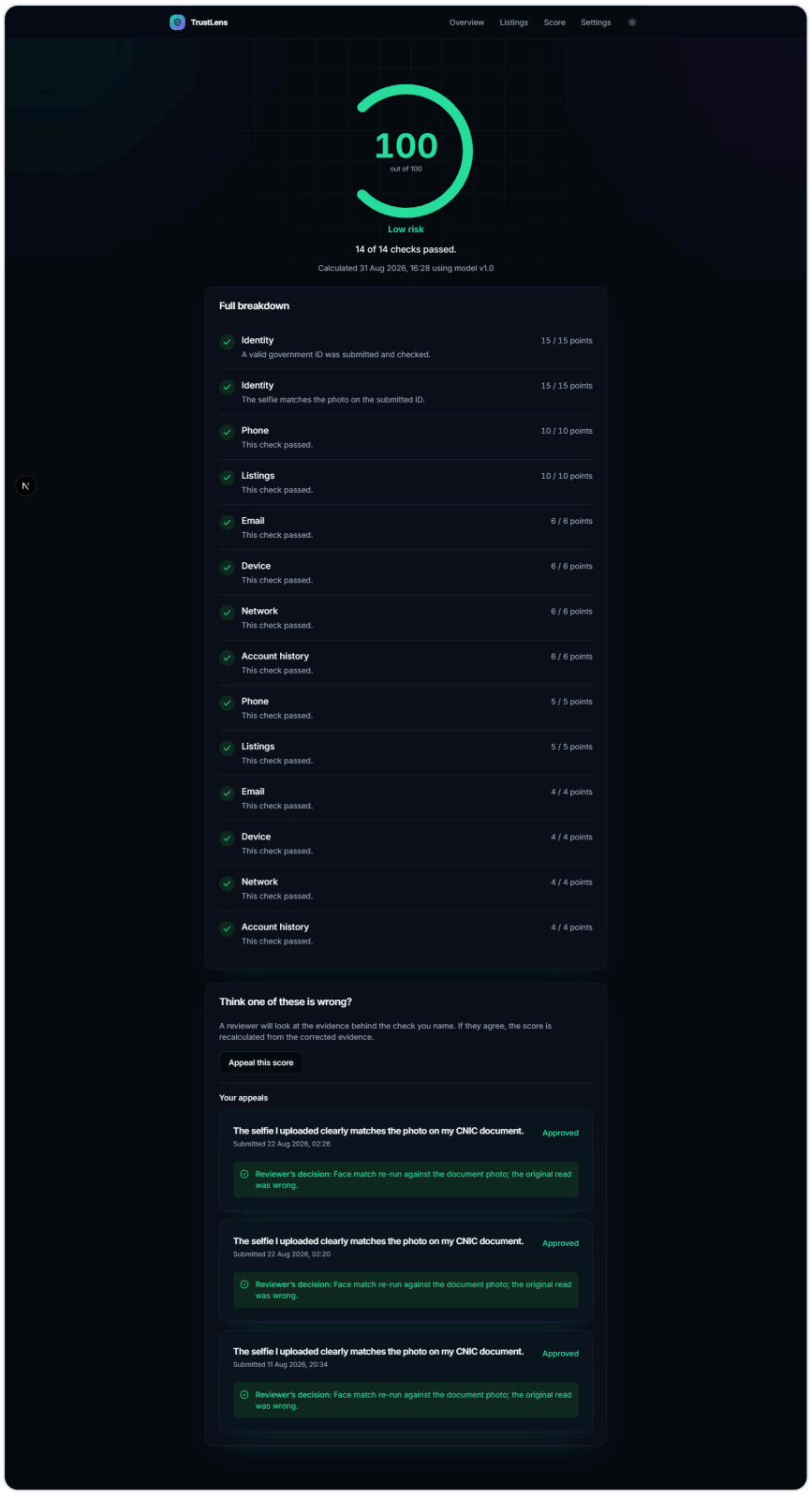


Figure 12a — A fully-verified seller's Trust Score page: the LOW-risk band, a category rollup, and every reason ranked by point impact.

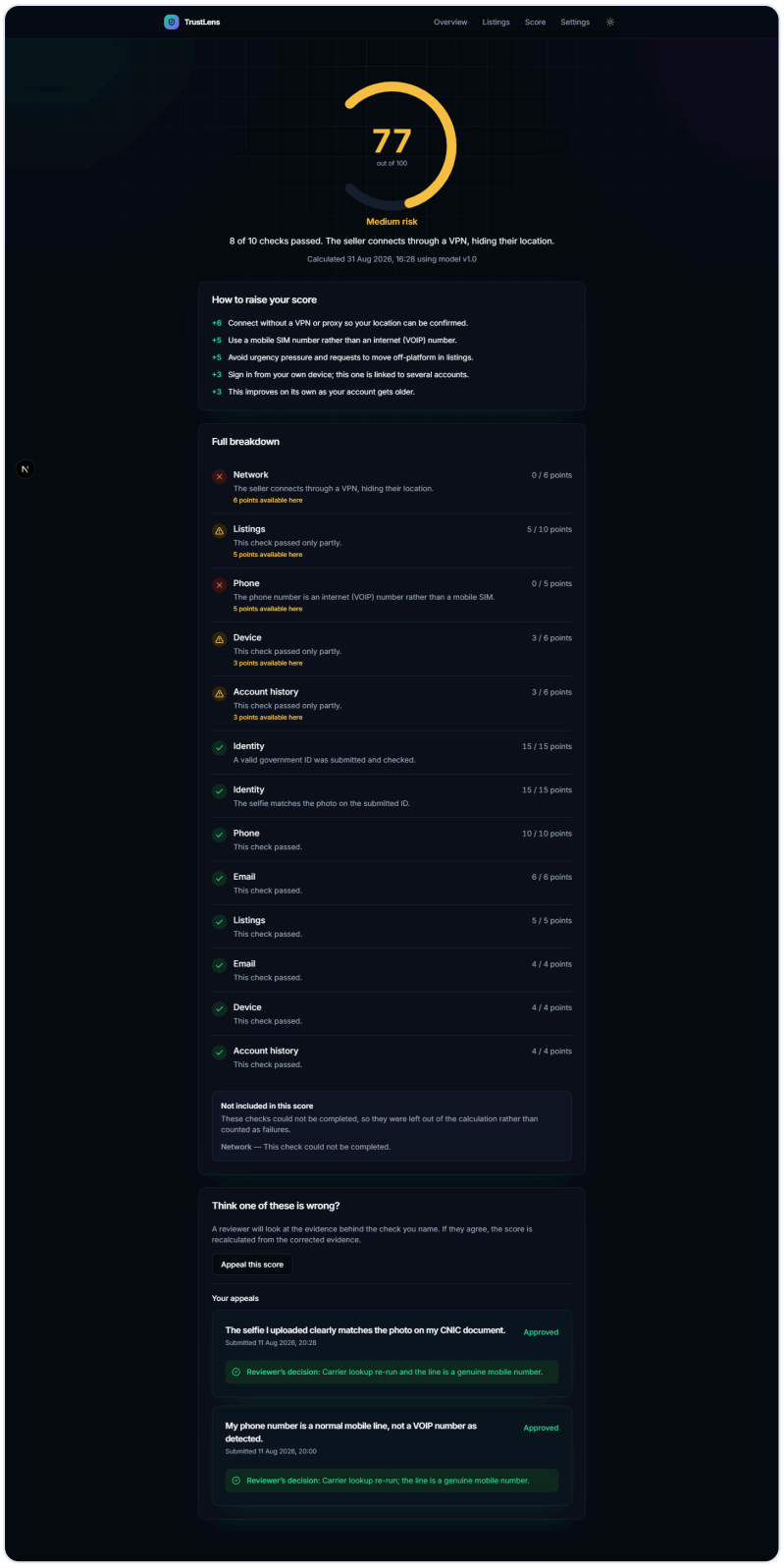


Figure 12b — A partially-verified seller in the MEDIUM-risk band — note that failed and unavailable signals are shown distinctly, not conflated.

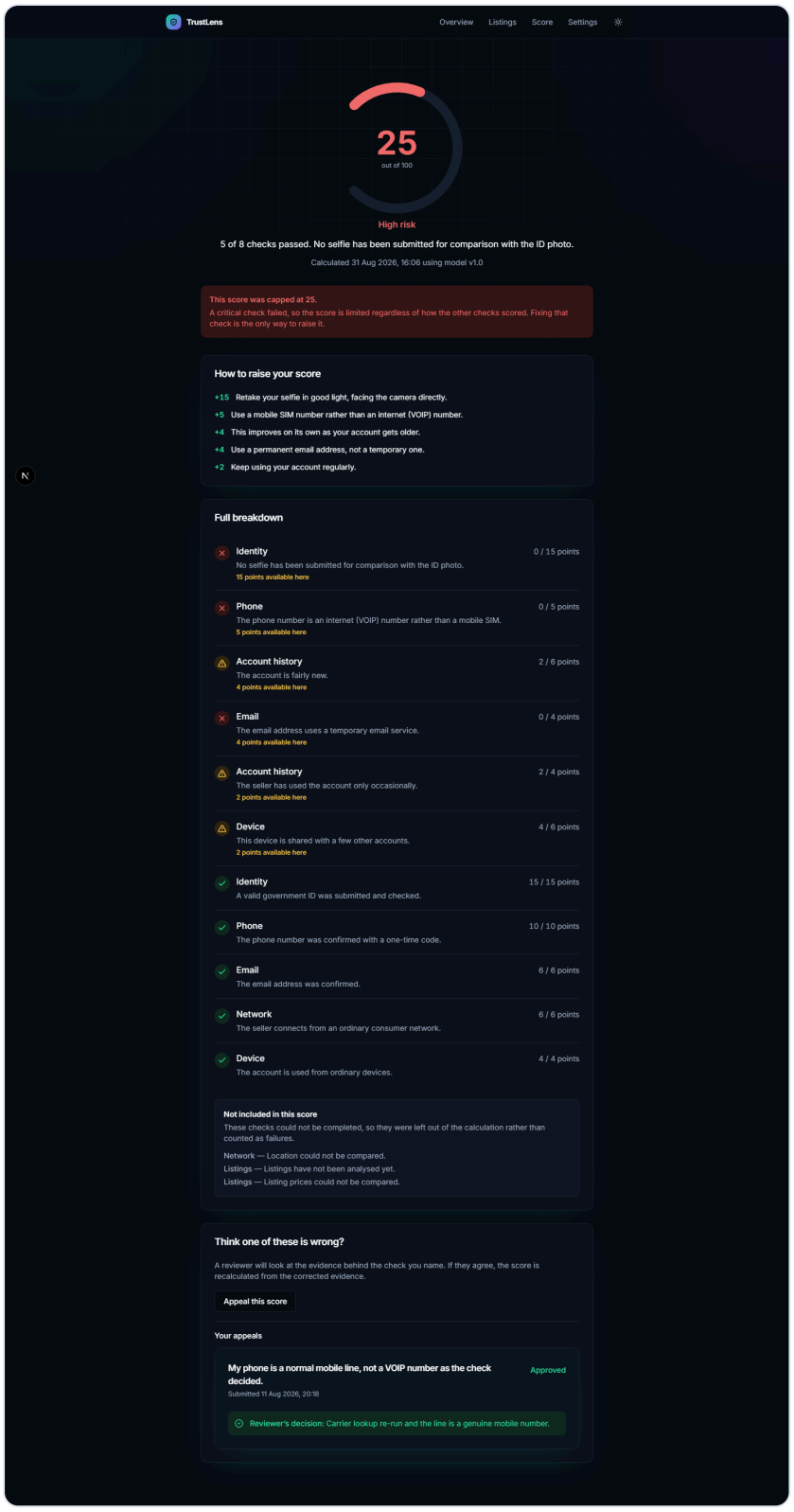


Figure 12c — A seller with a failed critical signal: the score is capped, and the interface states plainly that the cap — not the weighted average — is what is limiting the number, with a direct path to fix the specific failing check.

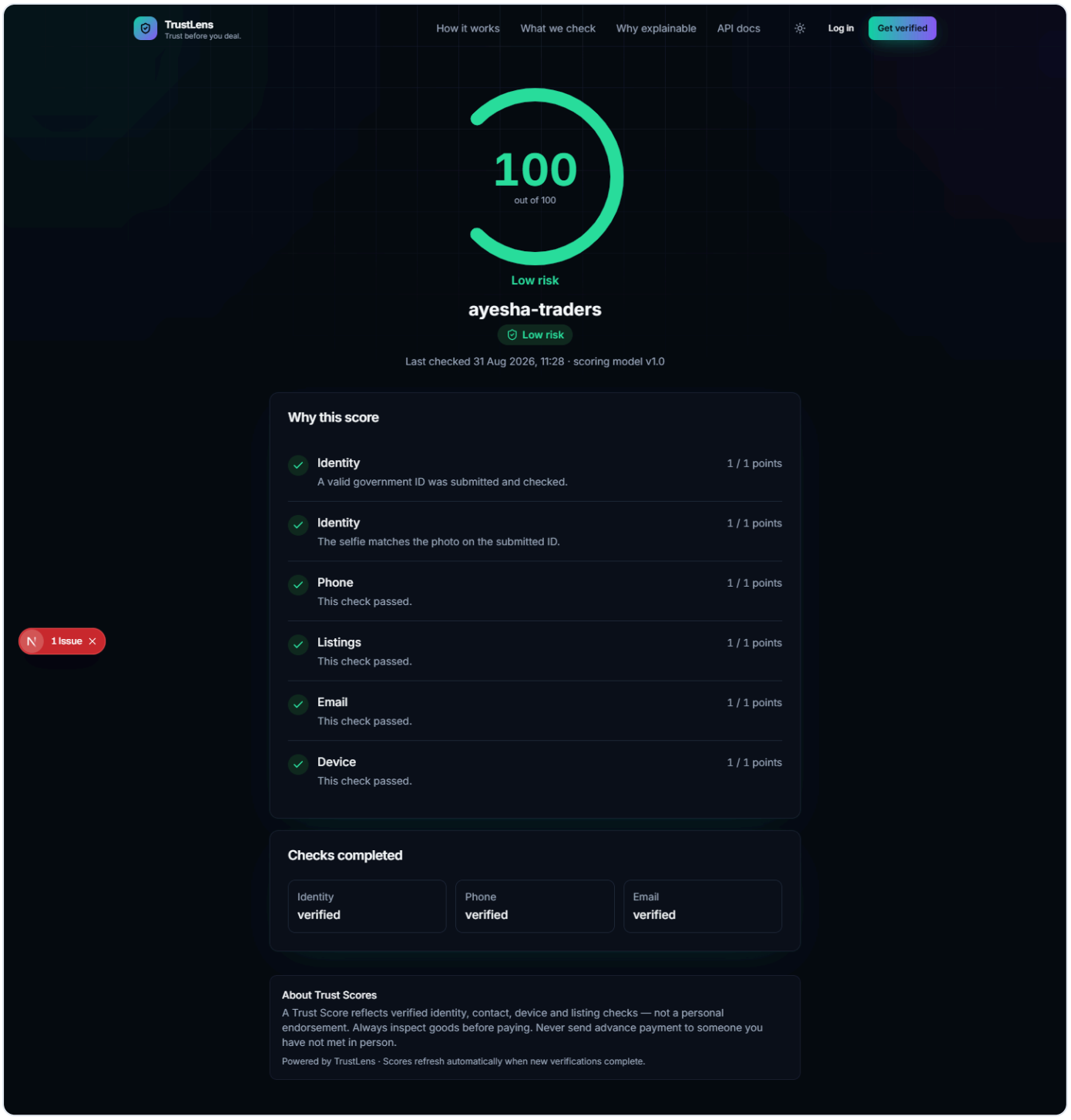


Figure 13 — The anonymous, no-account-required buyer view of a seller's score — the screen the product's entire value proposition depends on.

(Additional screenshots — the verification checklist, the mobile-viewport public page, and the admin review queue — are embedded in the User Manual, `06-User-Manual.md`, and referenced here by number where directly relevant to a results claim.)

7.2 Measured facts vs. design targets

Claim	Status
"Explainable Trust Score within seconds" (proposal's expected outcome)	**Corrected, not simply confirmed.** A score *read* is sub-second (cache hit, architecturally); full seller *enrollment* including identity verification takes 30–60 seconds. Both figures are stated because conflating them would misrepresent the system
330 automated tests passing	Measured, 22 August 2026, per the project's own CI history
50 API paths / 58 operations implemented	Measured against the live OpenAPI document
40-table schema, migrated and seeded	Measured
Public API latency target (p95 < 150ms)	**Design target, not yet load-tested against real infrastructure** — nothing is deployed
OCR/face-match accuracy on real documents	**No measurement exists** — explicitly out of scope for this report (§1.6)

7.3 Objectives-achieved scorecard (scored honestly against Chapter 1 §1.4)

Objective	Result
O1 — Verify seller identity	**Achieved** functionally (OCR + face match wired end-to-end, contributes to score); **not validated** against real documents
O2 — Verify contact channels	**Achieved**
O3 — Detect device/network fraud patterns	**Achieved** — device-clustering and VPN/proxy detection both feed the score
O4 — Analyse listing content with AI	**Partially achieved** — the LLM analyser works; the custom classifier works but is not live, given its measured cross-domain weakness (§6.3)
O5 — Produce one explainable score	**Achieved** — this is the objective the design most directly succeeds at
O6 — Support appeals	**Achieved**
O7 — Expose a public API	**Achieved** for score reads; partner-onboarding and webhook flows are implemented but less exercised
O8 — Operate at zero recurring cost	**Partially achieved** — the offline/local mode meets this fully; the public deployment target is designed and partly measured (worker RAM footprint: 323MB) but not yet live

5 of 8 objectives fully achieved, 3 partially achieved, 0 not attempted. No objective is marked achieved without the evidence cited in Chapter 6 or the SDD to support it.

7.4 Discussion — what the design choices turned out to matter for in practice

The decision to make hard caps and the UNAVAILABLE-signal exclusion explicit, rather than relying on a plain weighted average, was justified on paper in Chapter 4 — but its practical value only became fully visible while building the classifier evaluation in Chapter 6: had the classifier's weak cross-domain accuracy been wired directly into a weighted average, its false negatives would have quietly inflated real scammers' scores with no visible sign anything was wrong. Because the design already separates "evidence" from "score-setting," the fix was simply not enabling that evidence source yet — no scoring-engine change was needed, which is exactly the kind of resilience the architecture was intended to provide, discovered as a genuine (if slightly uncomfortable) validation of an earlier design decision rather than planned in advance.

Comparison with the related work in Chapter 2: unlike the commercial KYC vendors and the outcome-based reputation systems surveyed there, TrustLens's contribution is specifically the **combination** of attribute-based checks into one explainable, appealable number — not any single check in isolation, each of which individually resembles prior work.

7.5 Ethical and safety discussion

The system processes national identity documents and biometric face data for a real population. Design responses already in place: explicit versioned consent before any biometric processing, 90-day retention limits on raw documents, encryption of extracted sensitive fields, and — directly informed by the NIST demographic-fairness findings in Chapter 2 — a deliberate human-review band for borderline face-match results rather than a hard automatic threshold. The system explicitly does not claim to perform government identity verification, which would both overstate its actual capability and create a false sense of certainty for buyers.

Chapter 8 — Conclusion and Future Work

8.1 Conclusion

TrustLens demonstrates that combining several independent, individually-weak verification signals into one deterministically-scored, fully explainable Trust Score is both technically achievable within a two-student FYP's resources and materially more useful to a buyer than any single existing mechanism (a phone gate, a star rating) reviewed in Chapter 2. The core engineering — a clean-architecture domain package, an asynchronous AI pipeline kept out of the request path, and a scoring engine that treats AI output as evidence rather than verdict — is built, tested, and working. The project's honesty about what remains incomplete (admin panel, MFA, real-image accuracy validation, and the classifier's real weakness) is presented as a strength of this report, not a weakness of the project: an FYP that states its real limitations plainly is more defensible at viva than one that implies a completeness it has not achieved.

8.2 Contributions

1. A deterministic, versioned scoring engine that converts probabilistic AI evidence into an explainable, appealable score — with unavailable evidence excluded rather than penalised.
2. A working, end-to-end seller verification and appeal pipeline covering four independent signal types plus AI-assisted listing analysis.
3. A measured, honestly-reported cross-domain evaluation of a custom-trained scam classifier that surfaced a real domain-transfer failure before it could reach production scoring.
4. A documented, partly-benchmarked path to a $0/month public deployment (ONNX-based AI engines, measured image size and RAM footprint) for a resource-constrained student project.

8.3 Limitations (restated and expanded from Chapter 1)

No real CNIC/face accuracy data; the classifier is not production-ready for its actual target domain; the admin panel is incomplete; MFA is unimplemented; device fingerprinting and IP/VPN detection rely on heuristics with acknowledged accuracy limits rather than paid, fully authoritative data sources; no deployment is live; no independent security audit has been performed.

8.4 Future work

Priority	Item	Why this priority
Immediate	Collect real, consented CNIC/selfie test images and benchmark both engine pairs	Blocks every accuracy claim in the report; the single largest open item
Immediate	Complete the admin panel (seller search/detail, audit-log viewer)	Needed for full FR-ADMIN coverage and a credible demo of staff workflows
High	Label 300–500 real Pakistani marketplace listings and retrain the scam classifier	Directly fixes the measured cross-domain failure in Chapter 6
High	Implement MFA enrollment for staff accounts	Currently a documented security gap, not just an unimplemented nicety
Medium	Complete and rehearse the public deployment (Fly.io + Neon + Upstash + R2)	Designed and partly measured; needed for a live demo fallback
Medium	Reverse-image stolen-photo detection	The perceptual-hash seam already exists in the schema; named directly in the original problem statement
Lower	Mobile application, chat-based scam detection, direct marketplace integrations, seller reputation history	All explicitly out of scope for this FYP per the original proposal; each has an architectural seam already in place (SDD, and `docs/19-future-improvements.md`)

References

[1] P. Resnick and R. Zeckhauser, "Trust Among Strangers in Internet Transactions: Empirical Analysis of eBay's Reputation System," *Advances in Applied Microeconomics*, vol. 11, 2002.

[2] "State of the art of Trust and Reputation Systems in E-Commerce Context," arXiv:1710.10061, 2017.

[3] "A New Trust Reputation System for E-Commerce Applications," arXiv:1405.3199, 2014.

[4] S. Vidros et al., "Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset," *Future Internet*, vol. 9, no. 1, 2017. (Source of the EMSCAD dataset used as a training-corpus proxy in Chapter 5/6.)

[5] "Fake Job Detection and Analysis Using Machine Learning and Deep Learning Algorithms," presented at a peer-reviewed venue and indexed via ResearchGate, 2021.

[6] "AI-Based Document Forgery Detection Using OCR and Convolutional Neural Networks," SSRN preprint, 2025.

[7] "Identity Documents Authentication based on Forgery Detection of Guilloche Patterns," arXiv:2206.10989, 2022.

[8] "EdgeDoc: Hybrid CNN-Transformer Model for Accurate Forgery Detection and Localization in ID Documents," arXiv:2508.16284, 2025.

[9] "From Forgeries to Foundation Models: A Systematic Survey of Identity Document Attack and Detection," arXiv:2607.01442, 2026.

[10] P. Grother, M. Ngan, and K. Hanaoka, "Face Recognition Vendor Test (FRVT) Part 3: Demographic Effects," NIST Interagency Report NISTIR 8280, National Institute of Standards and Technology, 2019.

[11] "Designing Online Marketplaces: Trust and Reputation Mechanisms," *Innovation Policy and the Economy*, vol. 17, University of Chicago Press.

[12] FYP Proposal Submission Form No. 2, "TrustLens – AI-Powered Seller Trust Verification System for Online Marketplaces," Minhaj University Lahore, School of Software Engineering, 2026.

Appendix A — Full API reference

The complete endpoint table (50 paths, 58 operations, both `/api/v1` and `/public/v1`), request/ response shapes, status-code semantics, rate-limit table, and pagination/filtering/idempotency rules are in `04-SDD.md` §6 and, in full, `docs/06-api-design.md` of the engineering documentation set.

Appendix B — Full database schema summary

The complete 40-table catalogue with every column, type, constraint, and index is in `docs/05-database-design.md`. The core 13 tables carrying the product's explainability guarantee are diagrammed in `04-SDD.md` §3.1.

Appendix C — Full configuration reference

Every environment variable, grouped by domain (core, database, Redis, security/auth, AI providers, object storage, communications, rate limiting, observability, frontend), with per-environment differences, is in `docs/12-environment-variables.md`.

Appendix D — Revision history

Version	Date	Change
1.0	2026-08-31	Initial thesis manuscript authored against the built system for FYP submission